

Computer playing against a Random Adversary

Student Name: Zheng Rong Chua

Supervisor Name: Igor Razgon

Submitted as part of the degree of BSc Computer Science to the
Board of Examiners in the Department of Computer Sciences, Durham University

Abstract—Decision-making under uncertainty is a central challenge in artificial intelligence, particularly in environments characterised by stochastic dynamics and large state spaces. The puzzle game *Five-or-More* provides a suitable testbed for studying such problems, combining spatial reasoning, randomness, and long-term planning. This project aims to develop an AI search agent that maximises score in *Five-or-More*, while investigating the impact of heuristic evaluation design and search enhancements on decision-making performance. A complete game environment was implemented in Python, and a range of domain-informed heuristic functions were constructed and evaluated using a greedy search framework. Additional techniques, including look-ahead search, Monte Carlo simulation, and alpha-beta pruning, were incorporated to enhance decision quality under uncertainty. Results show that heuristic design plays a critical role in agent performance, with combined and adaptive heuristics outperforming simpler approaches. The integration of heuristics with search-based methods further improves performance, demonstrating the importance of balancing evaluation quality with search depth. Overall, this work highlights the effectiveness of structured heuristic design in developing interpretable and efficient AI systems for stochastic decision-making problems.

Index Terms—Decision problem, Heuristics design, Monte Carlo, Stochastic processes



1 INTRODUCTION

THIS project focuses on the design and development of an artificial intelligence agent to play the puzzle game *Five-or-More*, with the aim of investigating decision-making strategies under uncertainty.

1.1 Background / Context

Decision-making under uncertainty is a fundamental challenge in artificial intelligence, arising in environments where outcomes are influenced by both stochastic processes and complex system dynamics. Such problems are pervasive across a wide range of domains, from game playing AIs, to financial decision-making and logistics planning, to everyday human behavior. In these settings, agents must make informed decisions based on incomplete or probabilistic information, often balancing short-term gains against long-term consequences.

A common approach to addressing such problems in artificial intelligence is through the use of search algorithms guided by heuristics, i.e. evaluation functions. These methods operate by generating possible future states and assigning each a value that reflects its desirability. The effectiveness of such systems depends critically on both the structure of the search process and the quality of the evaluation function used to guide it. In particular, heuristic evaluation functions play a central role in enabling efficient decision-making by providing tractable approximations of long-term outcomes [1], [2].

The development of heuristic-guided search methods can be traced from early applications in combinatorial optimization to broader use in complex and uncertain environments. In operations research, heuristics were introduced to address large-scale problems such as the Traveling Salesman Problem, where exhaustive enumeration is infeasible, enabling efficient

navigation of exponentially large solution spaces through approximate evaluation [3].

Building on these foundations, heuristic approaches became central to game-playing artificial intelligence, where evaluation functions guide search through large game trees in domains such as chess. Similar principles extend to robotics and planning, where heuristics enable efficient exploration of high-dimensional state spaces [4].

Beyond these areas, heuristics are widely used in domains characterized by both complexity and uncertainty. In financial and trading contexts, decision-making is often supported by models that incorporate domain knowledge and heuristic approximations of risk and return, requiring agents to balance expected profit against exposure to uncertainty [5], [6]. Comparable approaches appear in sports analytics and betting models [7].

Heuristic methods have further been adopted across scientific and engineering disciplines. In computational biology, they manage exponential complexity in tasks such as sequence alignment and structure prediction [8], while in control systems they provide practical solutions when precise models are unavailable [9]. Similarly, in natural language processing, heuristic scoring functions have historically guided translation and decoding processes [10].

Across these diverse applications, the underlying principle remains consistent: the use of tractable evaluation functions to approximate long-term outcomes, enabling effective decision-making in environments characterized by large state spaces, uncertainty, and computational constraints.

1.2 Problem

For our exploration, the puzzle game *Five-or-More* provides a suitable controlled testbed for studying these challenges.

The game is played on a 9×9 grid in which colored tiles, drawn from six possible colors, are randomly generated and placed over time. At each step, the player moves a tile to an empty location, subject to path constraints, with the aim of forming lines of five or more matching colors to score points. The game's primary challenge arises from a random adversarial tile spawning mechanism. Three new color tiles are spawned at random locations after each turn unless a line clear is achieved, creating a natural incentive to prioritize clears in order to both maximize score and halt spawning to extend survivability. The game terminates when the board becomes full, and the total points tallied for evaluation.

Formally, the game represents a single-agent, fully observable, stochastic decision-making problem, where outcomes depend on both the agent's actions and random environmental transitions. The stochastic spawning of tiles introduces uncertainty, while the spatial and combinatorial structure of the board creates a large and evolving decision space. Although the environment does not act strategically, its randomness can be interpreted as introducing adversarial-like constraints that progressively disrupt board structure. At each step, the agent must select a move from a set of feasible actions, anticipating both immediate outcome and future stochastic transitions. Effective strategies must hence balance immediate rewards, such as clearing lines, with long-term positioning and risk management. As the game progresses, the accumulation of randomly placed tiles increases the difficulty of maintaining coherent structures.

Adding to the uniqueness of this problem, the game provides limited preview information in the form of upcoming tile colors, excluding their future spawn locations. This allows the agent to incorporate partial knowledge of future states into its decision-making, while still operating under stochastic uncertainty.

Overall, these characteristics make the game well-suited for exploring and analyzing decision-making strategies in stochastic environments.

1.3 Motivation

While *Five-or-More* is a relatively simple game, its underlying structure reflects a broader class of decision-making problems encountered in real-world systems, motivating its study.

This is supported by its interpretation across multiple theoretical frameworks, highlighting its relevance across several fields. Mathematically, it can be formulated as a fully observable stochastic decision process, or more specifically a Markov Decision Process (MDP), in which the next state depends on the current state, chosen action, and stochastic transition dynamics [11], [12]. From an information-theoretic perspective, it can be viewed as a process of managing entropy within the board state, where random tile generation introduces disorder and effective strategies maintain structured configurations [13]. From a decision-theoretic or financial perspective, the problem corresponds to balancing short-term gains (profit maximization) against long-term survivability (risk management) under stochastic uncertainty [14].

A key motivation for studying this game lies in its distinction from studies on existing traditional game-playing artificial intelligence. In classical domains such as chess,

Go, or poker, agents operate against strategically rational adversaries, where uncertainty arises from strategic interaction. In contrast, *Five-or-More* introduces a stochastic adversarial environment that imposes random constraints without strategic intent. This form of uncertainty more closely resembles real-world environments, particularly financial markets, where external agents and events influence outcomes in unpredictable and often non-rational ways [5].

In this trading context, the random spawning of tiles can be interpreted as analogous to external participants or market forces entering a system. While certain information may be known in advance, such as the set of upcoming tile colors, their exact impact remains uncertain. The agent must therefore make decisions based on partial foresight, anticipating possible future configurations and weighing stochasticity into its objectives.

The game can therefore be viewed as an abstract representation of such systems. The grid acts as a constrained environment in which decisions occupy and modify available resources, while the stochastic spawning process alters the landscape over time. Strategies for managing space, anticipating constraints, and adapting to uncertainty thus become directly analogous to broader decision-making principles.

As a result, studying decision-making within *Five-or-More* provides insight into general approaches for handling uncertainty. Its simplicity allows these concepts to be examined in a controlled setting, while still retaining key characteristics of more complex real-world systems.

1.4 Project Objectives

This project is structured around two primary objectives:

- (i) **Project Deliverable:** The development of AI search agents, exploring different heuristic design and search enhancements, capable of achieving strong performance in the game *Five-or-More*.
An intermediate objective is to investigate the integration of machine learning approaches with heuristics, specifically deep reinforcement learning strategies inspired by modern game-playing AI systems. An advanced objective of the project is the integration of the AI system into a human-interactive gameplay framework, enabling the agent to provide real-time decision support and guidance during gameplay.
- (ii) **Research Objectives:** To investigate how heuristic evaluation design and search-based enhancements influence decision-making and performance. In particular, the project seeks to answer the following research questions:
 - How does the design of heuristic evaluation functions affect the performance and behavior of an AI search agent?
 - To what extent do search-based enhancements, such as lookahead and Monte Carlo simulation improve decision-making in a stochastic environment?

2 RELATED WORK

The concepts explored in developing an AI agent for the game *Five-or-More*, as introduced previously, can be observed across multiple disciplines and practical applications. This section expands on this by surveying existing work related to the core problems and concepts underlying this task, including works from areas such as game-playing artificial intelligence, financial decision-making, information theory, and generally decision-making under uncertainty.

In particular, through these bodies of literature, we examine the key components and terminology that underpin both the problem formulation and the solution approaches explored in this project—namely state-space modeling, heuristic evaluation function design, classical search optimization techniques, stochastic search methods such as Monte Carlo simulation, and the trade-off between computational efficiency and decision performance.

The following subsections review these areas in turn, highlighting how existing methods address similar challenges, and identifying the gaps and distinctions that motivate the approach taken in this project.

2.1 Problem Abstraction and State-Space Representation

A fundamental step in applying artificial intelligence to decision-making problems is the abstraction of the environment into a formal state-space representation. In this framework, a problem is modeled as a set of states, a set of actions defining transitions between states, and an objective function that evaluates the desirability of outcomes. Such representations form the foundation of many classical and modern AI systems, enabling agents to reason about possible future configurations in a structured and computationally tractable manner [1], [15], [16]

State-space modeling has been widely adopted across a range of domains. In game-playing artificial intelligence, environments such as chess and Go are naturally represented as discrete state spaces, where board configurations define states and legal moves define transitions, allowing search algorithms to explore future outcomes [1], [4]. More generally, similar representations are used in robotics and planning, where agents navigate high-dimensional spaces under constraints. In financial and trading contexts, analogous formulations arise in sequential decision-making models, where states encode market conditions and actions correspond to investment decisions, with transitions governed by stochastic dynamics [12].

Accurate state-space abstraction is therefore a crucial first step in enabling an agent to correctly process and navigate the problem.

2.2 Heuristic Evaluation Function Design

Heuristic evaluation functions form a central component of search-based artificial intelligence systems, providing a mechanism for estimating the desirability of states without requiring exhaustive exploration of the search space. In many complex environments, it is computationally infeasible to evaluate all possible future outcomes, and therefore heuristic functions are used to approximate the long-term value of a given state based on its current configuration [1], [2].

Formally, an evaluation function assigns a numerical score to a state, reflecting its relative quality with respect to the agent’s objective. In game-playing systems, this often corresponds to estimating the likelihood of achieving a favorable outcome from a given position. Such functions are typically applied at the leaves of a truncated search tree, enabling the agent to compare candidate actions and select those leading to the most promising states. Consequently, the performance of a search-based agent depends heavily on the quality and structure of its evaluation function [1], [2].

A key characteristic of heuristic evaluation functions is their domain dependence. Unlike exact evaluation methods, which are only possible in fully solved or trivial environments, heuristic functions must be designed to capture the structural properties and constraints of a specific problem. In classical domains such as chess, evaluation functions apply a weighted combination of features such as material balance, positional control, and piece activity. Similarly, research in general game playing has explored methods for constructing heuristic functions that generalize across different games, though performance remains highly sensitive to how well the heuristic captures domain-specific dynamics [17].

This highlights an important limitation: there is no universal or “one-size-fits-all” heuristic that performs optimally across all environments. Instead, effective heuristic design requires an understanding of the underlying mechanics and constraints of the problem domain. Empirical studies have shown that variations in heuristic structure can significantly influence the behavior and performance of search algorithms, even when the underlying search method remains unchanged [18].

Furthermore, heuristic functions are often refined through parameter tuning or optimization techniques, including gradient-based methods and evolutionary approaches [19]. The choice of parameters can significantly influence agent performance, making tuning itself a non-trivial challenge in heuristic design.

In stochastic environments, the role of heuristic evaluation becomes even more critical. This appears in domains such as poker and financial trading, where uncertainty, incomplete information, and long-term reward considerations play a central role. In poker, evaluation functions must account for hidden information and probabilistic outcomes, requiring strategies that balance immediate gains with expected future value [20].

These complexities are further amplified in financial markets, where decision-making is influenced by a combination of stochastic dynamics, external agents, and non-stationary conditions. In such settings, it is often impractical to rely on purely data-driven models alone. Instead, practitioners frequently incorporate domain expertise into the design of evaluation and decision-making frameworks, constructing heuristic representations of factors such as risk, momentum, liquidity, and market structure [21].

For example, alternative data sources such as satellite imagery have been used to estimate physical indicators like oil storage levels by analyzing the shadow patterns of oil tanks, providing indirect signals about supply conditions and future price movements [22]. Such approaches serve as heuristic proxies for otherwise unobservable market variables, translating complex real-world phenomena into

tractable features that can inform decision-making.

In the context of this project, heuristic evaluation functions are treated as a primary area of exploration. Rather than assuming a fixed evaluation strategy, this work investigates a range of domain-informed heuristics designed to capture different aspects of the game state. By analyzing how these heuristics influence the performance of an AI search agent, this project aims to better understand the relationship between heuristic design and decision-making in stochastic environments.

2.3 Alpha-Beta Pruning

Classical search paradigms such as minimax and its optimisation via alpha-beta pruning have been widely used in adversarial domains, particularly in board games such as chess [1]. Alpha-beta pruning improves the efficiency of minimax by eliminating branches of the search tree that cannot influence the final decision, allowing deeper search within the same computational constraints [23].

The effectiveness of such methods depends heavily on the quality of the evaluation function used to approximate non-terminal states. However, these approaches are typically designed for deterministic, fully observable environments with rational adversaries. In contrast, stochastic environments such as *Five-or-More*, where outcomes are influenced by random processes / adversary rather than an optimal opponent, present additional challenges that limit the direct applicability of purely adversarial search methods. Hence, while alpha-beta pruning has proven highly effective in deterministic, adversarial settings, it is unsuitable as a modeling framework for this game.

2.4 Lookahead and Monte Carlo Tree Search

In contrast to classical adversarial search methods, lookahead techniques and Monte Carlo Tree Search (MCTS) has been demonstrated to provide a more flexible framework for decision-making in complex and stochastic environments. Lookahead search extends the depth of exploration by simulating future states beyond immediate actions, allowing agents to incorporate longer-term consequences into their decision-making. However, as the depth of lookahead increases, the computational cost grows exponentially, making exhaustive evaluation infeasible in large state spaces.

Monte Carlo Tree Search addresses this limitation by using stochastic sampling to approximate the value of actions. Rather than evaluating all possible future states, MCTS performs repeated simulations (rollouts) from the current state, using random or semi-guided policies to estimate the expected outcome of different actions [24]. This allows the algorithm to focus computational effort on the most promising regions of the search space while maintaining scalability in complex environments.

MCTS has achieved notable success in game-playing domains, particularly in systems such as AlphaGo and AlphaZero, where it is combined with learned evaluation functions to guide exploration and achieve superhuman performance [4], [25]. Recent research has also explored the application of Monte Carlo Tree Search in financial decision-making, where the problem structure shares similarities with stochastic sequential environments. For instance, MCTS has

been applied in portfolio optimization under uncertainty, demonstrating how simulation-based search can be used to evaluate future market scenarios and guide sequential investment decisions [26], [27]. In such settings, MCTS enables agents to reason about distributions of possible outcomes rather than relying on deterministic predictions, a property that is essential in environments characterized by volatility and incomplete information [28].

The applicability of Monte Carlo methods across both games and real-world domains highlights their effectiveness in handling uncertainty and large search spaces. In particular, their ability to approximate expected outcomes through sampling makes them well-suited for problems where explicit evaluation functions may be difficult to construct or insufficient on their own. This motivates their use, and the exploration of their impact on performance in conjunction with heuristic evaluation in our task, allowing for a balance between structured domain knowledge and probabilistic exploration.

2.5 Machine Learning Approaches

In recent years, machine learning, particularly deep reinforcement learning, has become a dominant approach for solving complex decision-making problems. Systems such as AlphaGo and AlphaZero combine deep neural networks with search algorithms to learn evaluation functions and policies directly from data, enabling them to achieve superhuman performance in a range of game environments [4], [25]. These approaches replace manually designed heuristics with learned value functions, allowing agents to generalize from experience and adapt to complex patterns within the environment.

Despite their success, such methods are often computationally intensive and require large amounts of training data. Furthermore, the resulting models are typically difficult to interpret, as the learned evaluation functions operate as black-box approximations of value. This can limit their applicability in domains where transparency, interpretability, or computational efficiency are important considerations.

As a result, alternative approaches have explored combining domain knowledge with learning-based methods. In these hybrid frameworks, structured features or heuristics are first constructed based on domain understanding, and machine learning is then used to parameterize or learn nonlinear relationships these representations [29], [30], [31]. Such approaches aim to retain interpretability and efficiency while benefiting from data-driven adaptation in stochastic environments.

These approaches highlight a range of strategies for addressing decision-making under uncertainty, from fully learned models to hybrid methods that integrate domain knowledge with data-driven adaptation. Collectively, they inform the design space for constructing effective evaluation functions and decision-making frameworks in complex stochastic environments.

2.6 Managing Entropy

The stochastic nature of *Five-or-More* introduces a natural interpretation of the problem in terms of managing uncertainty within the state space. From an information-theoretic

perspective, entropy is defined as a measure of uncertainty over possible system configurations [13]. In this context, the random generation of tiles increases uncertainty by introducing unpredictable state transitions and progressively constraining the set of feasible future actions.

Effective decision-making can therefore be viewed as a process of entropy management, where the agent seeks to maintain structured and flexible configurations while mitigating the impact of stochastic events. In practice, this involves preserving optionality within the board, maintaining coherent tile arrangements, and anticipating the effects of random tile placements to avoid highly constrained or unstable states.

Similar considerations arise in domains such as financial decision-making, where uncertainty and variability play a central role. In such settings, uncertainty is not managed directly through entropy, but through related concepts of risk and variability. These are formalized using measures such as volatility (the variance of returns), drawdown (the magnitude of losses from peak value), and tail risk (the probability of extreme adverse outcomes) [32], [33].

2.7 Evaluation Metrics

The evaluation of decision-making systems varies depending on the objectives and characteristics of the domain. In many game-based environments, performance is commonly assessed using metrics such as the maximum score achieved or the average score over multiple runs. The maximum score reflects the agent’s peak capability under favorable conditions, while the average score provides a measure of consistency and robustness across stochastic outcomes.

This distinction becomes particularly important in environments characterized by uncertainty. In stochastic settings, an agent may occasionally achieve very high scores due to favorable random events, even if its underlying decision-making strategy is not consistently effective. As a result, relying solely on maximum performance may not provide a complete representation of an agent’s overall quality.

In contrast, real-world decision-making domains such as financial trading place greater emphasis on consistency and risk management. In these settings, strategies are typically evaluated based on their expected return and variability, where stable and predictable performance is often preferred over high but volatile returns. Monte Carlo methods are frequently used in such domains to estimate distributions of possible outcomes, allowing for evaluation based on both expected performance and risk [28].

2.8 Computation Complexity vs Decision Performance

A central challenge in search-based decision-making is the trade-off between computational complexity and decision quality. In many combinatorial environments, the size of the search space grows exponentially with the number of possible actions and future states, making exhaustive evaluation infeasible. This is particularly evident in game-playing domains such as chess and Go, where the branching factor and search depth result in an immense number of possible game configurations [1]. Similar challenges arise in stochastic decision-making problems, where randomness further increases the effective size of the state space.

To address this, a wide range of techniques have been developed to improve computational efficiency while maintaining decision performance. In classical game-playing systems, efficient state representations, such as bitboards in chess engines, enable faster computation through compact encoding and parallelized operations [34].

In stochastic and high-dimensional domains, sampling-based approaches provide an alternative means of managing complexity. Monte Carlo Tree Search, for instance, selectively expands the most promising regions of the search space rather than exploring all possible actions. Practical implementations often further restrict computation by considering only a subset of candidate actions (e.g. top- k moves ranked by heuristic evaluation), allowing deeper or more refined analysis within limited computational budgets. Similarly in financial decision-making contexts, approximation methods—including Monte Carlo simulation, scenario sampling, and heuristic filtering—are commonly used to balance computational tractability with decision quality [28].

These approaches highlight a fundamental principle: improving decision performance does not necessarily require exhaustive search, but rather effective allocation of computational resources toward the most informative parts of the decision space. This trade-off is central to the design of intelligent agents operating in complex and uncertain environments.

2.9 Random Adversary

As introduced previously, a distinguishing feature of *Five-or-More* is its formulation as a stochastic environment driven by a *random adversary*. Rather than optimizing against a rational opponent’s strategy like in chess or poker, the agent must combat against non-strategic, stochastic dynamics by reasoning over distributions of possible outcomes and manage the accumulation of uncertainty over time. This places *Five-or-More* within a less explored class of AI Game problems. Hence, we turn towards real-world domains such as financial markets which contain analogous challenges.

In trading and portfolio optimization, agents operate under uncertainty where market forces and external participants do not always behave rationally, often deviating from purely rational models [5]. To address such challenges, financial and quantitative methods often rely on probabilistic modeling and simulation-based approaches, alongside risk-aware evaluation frameworks. Techniques such as Monte Carlo simulation are widely used to evaluate future scenarios, estimate expected returns, and assess risk under uncertainty [28]. These approaches shift the focus from deterministic optimization to expectation-based reasoning, enabling agents to operate effectively in environments where precise prediction is infeasible.

This perspective further motivates the exploration of Monte Carlo Tree Search and risk-aware evaluation strategies, alongside heuristics, within our context of *Five-or-More*.

3 METHODOLOGY

3.1 Problem Formal Abstraction

The first stage of the project involved abstracting the game *Five-or-More* into a formal representation that could be processed by an artificial intelligence agent. This required the game to be modeled as a state-space search problem, in which each board configuration corresponds to a state, each legal tile movement defines an action, and the game mechanics determine the resulting state transition.

3.1.1 State Representation

As illustrated in Fig. 1, our initial abstraction of the game represents the board as a 9×9 matrix implemented using a NumPy array, where each entry stores the contents of a cell. An empty cell is encoded by 0, while occupied cells are represented by integer values corresponding to tile colors. This representation provides a clear and intuitive encoding of the board state, while allowing matrix operations and coordinate-based indexing to be performed directly in Python.

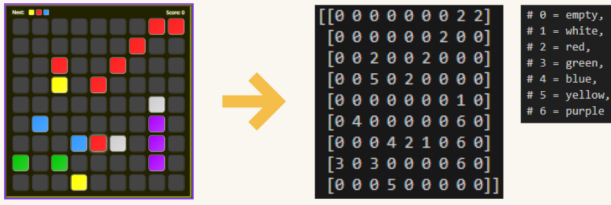


Fig. 1. Example 9×9 board representation used in *Five-or-More*, where each cell stores an integer corresponding to tile colour, and 0 represents an empty cell.

3.1.2 Legal Actions and Reachability

As demonstrated in Fig. 2, at each turn, the player may select one occupied tile and move it to an empty destination cell, provided that a valid path exists between the two positions. In particular, *Five-or-More* imposes a path constraint: a tile may only be moved if it can travel through a connected sequence of empty cells.

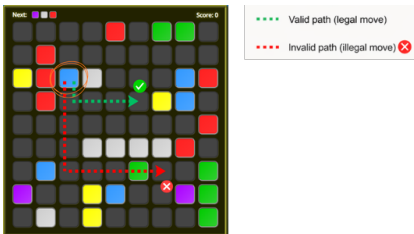


Fig. 2. Example Path Restriction game mechanic

To model this, legal action checking is implemented using Breadth-First Search (BFS). Given a source and destination cell, BFS explores neighbouring empty cells in the four cardinal directions until either the destination is reached or all reachable positions have been exhausted. If the destination is found, the move is considered legal; otherwise it is rejected. BFS was chosen because it provides a complete and efficient method for determining reachability on an unweighted grid, making it well-suited to the movement rules of the game.

3.1.3 State Transitions and Game Mechanics

Once a legal move is selected, the tile is transferred from its source cell to the destination cell, producing an intermediate board state. This transition is implemented deterministically by updating the matrix representation: the destination cell takes the value of the source tile, and the source cell is set to empty.

Following the move, as illustrated in Fig. 3, the board is evaluated for the formation of lines of five or more tiles of the same color. This is performed by a directional scanning procedure centered on the moved cell. For each of the four principal directions (horizontal, vertical, and the two diagonals), contiguous tiles of the same color are identified by extending outward in both positive and negative directions. If the total length of any such line meets or exceeds five, all tiles belonging to that line are marked and set to empty simultaneously using a boolean mask over the board. The lengths of cleared lines are recorded for scoring.

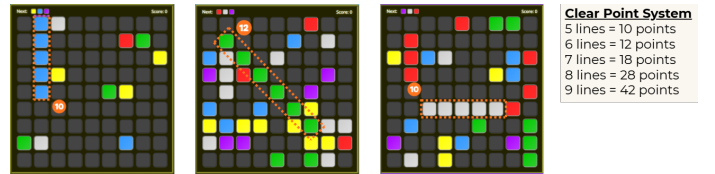


Fig. 3. Examples of Game Clear Mechanic and Point System

As illustrated in Fig. 4, if no line is cleared as a result of the player's move, the environment performs a stochastic transition by spawning up to three tiles, subject to the number of available empty cells. The colors of these tiles are selected randomly and predetermined prior to the move and stored in `next_colors`, replicating the preview information available to the agent.

To perform the spawn, all empty cells on the board are first identified. A subset of these positions is then selected uniformly at random for spawn. This ensures that while the agent has partial knowledge of future tile colors, their spatial placement remains stochastic.

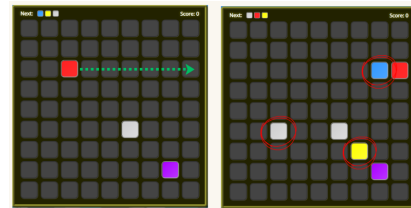


Fig. 4. Example Demonstration of Game Random Spawn Mechanic

After spawning, in case the spawned tiles complete lines of five or more, each newly placed tile is individually evaluated for potential line formation using the same directional scanning procedure. Any resulting lines are removed and scored accordingly, producing the next state of the game.

The resulting transition model is therefore hybrid in nature: player/agent actions are deterministic, while tile generation introduces stochasticity. Through this formulation, the core mechanics of the game—including tile spawning, line clearing, and score accumulation—are replicated within the environment.

3.2 AI Search Pipeline

The AI agent developed in this project is designed according to the standard pipeline used in search-based artificial intelligence, as illustrated in Fig. 6.

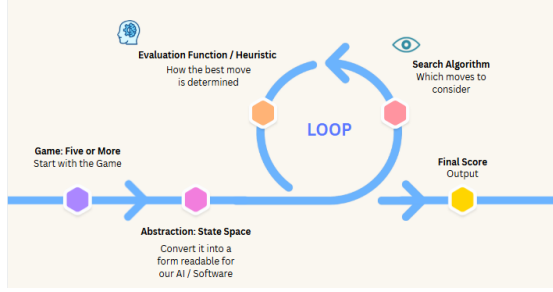


Fig. 5. Diagram of AI Search Pipeline

Within this pipeline, our main areas of exploration therefore focus on the design of heuristic evaluation functions and the integration of search-based enhancements, including Monte Carlo Tree Search and alpha-beta pruning.

3.3 Heuristic Evaluation Function Design

To effectively explore and isolate the effect of heuristic design, initial experiments were conducted using a simple greedy (depth-1) search strategy. In this setting, all legal moves from the current state are generated, and the resulting states are evaluated solely based on the heuristic function. This allows the impact of different evaluation strategies to be analyzed independently.

3.3.1 Domain-Informed Heuristic Design

A range of heuristic functions were developed based on domain-specific insights into the mechanics of *Five-or-More*. These were informed by game-play experience, accumulated domain knowledge, experimentation and observations in simulation runs by the agent.

All heuristics developed are summarised in Table 1, where their formulation, underlying intuition, and key benefits/limitations are reviewed and presented. This structured comparison provides a clear overview of the design considerations and trade-offs associated with each approach. Each of these heuristics captured a different aspect of the game and element of what makes a good move in differing scenarios.

TABLE 1
Summary of Heuristic Evaluation Functions Designs

Heuristic	Formulation / Review
Foreword	Each heuristic evaluates legal moves by applying the move, clearing any 5-or-more line created by the moved tile, and assigning a score based on the resulting board state.
[1] Trivial Heuristic (No. Empty)	Formulation: Counts the number of empty cells in the resulting board state and favours states with more empty cells. Consequently, it selects moves that reduce board occupancy, which in the context of <i>Five-or-More</i> corresponds to moves that produce line clears. Review: While aligned with the objective of prolonging survivability by avoiding full occupancy of the board (i.e. end-game scenario), it does not explicitly encourage line formation and hence point scoring.

Heuristic	Formulation / Review
[1] Clear Bonus	Formulation: Assigns move evaluations score, proportional to the length of lines cleared and game points gained. Review: This directly incentivizes scoring actions and prioritizes immediate clears when available. However, it does not provide a mechanism for setting up lines and is only effective when near-complete configurations (e.g. lines of four) arise incidentally through chance or prior moves.
[3] Line Potential	Formulation: From game-play observation, forming lines of five requires progressively building longer sequences of matching tiles (1 → 2 → 3 → ...). This heuristic captures this by scanning all rows, columns, and diagonals to identify maximal contiguous sequences of the same color. Sequences of length 2, 3, 4, 5, 6... are assigned increasing scores (according to a predefined mapping), and the total value is obtained by summing these across all directions. Moves are therefore preferred if they increase the number or length of these sequences, encouraging incremental line formation. Review: While it follows the common line creating strategy, it aggressively prioritizes strictly contiguous sequences and does not account for other factors such as gap-based setups or overall board structure, thereby limiting the range of line-formation strategies and potentially non-actionable configurations. Furthermore, it only indirectly promotes line clears through favoring longer sequences; this is resolved by integrating the cleared bonus heuristic, enabling both the setup and prioritization of scoring moves through clears.
[4] Gap Line Potential	Formulation: Awards higher scores to configurations containing five-cell windows made up of tiles of a single colour and gaps (e.g. R _ R R R), which can be completed in future moves. Higher scores are assigned to windows with more same colored tiles already present. This encourages moves that create or preserve near-complete line structures, even when tiles are not strictly adjacent, unlike as with the line potential heuristic. Review: While it presents an alternative strategy for line formation, it similarly requires cleared bonus heuristic to prioritize clears. Additionally, as a heuristic, restricting to any singular line formation strategy presents limitations, which motivates the conception of a composite heuristic later on.
[5] Clustering	Formulation: Inspired by clustering methods used to reduce fragmentation and impose structure in data representations [35], this heuristic measures the spatial proximity of tiles of the same color via centroid-based dispersion. Moves are then evaluated higher if they reduce the spread of same-color tiles, encouraging more compact groupings. Review: While it promotes structural grouping, it does not care of line structures and ignores path constraints, meaning spatial proximity does not guarantee that tiles can be connected or moved effectively in <i>Five-or-More</i>. This evidently proved ineffective as discussed in results and evaluation.
[6] Path-Based Mobility	Formulation: Estimates mobility using Breadth-First Search (BFS) to count the number of reachable and useful positions each tile can be moved to, where the condition of usefulness is defined by a position adjacent to another tile of the same color. Review: This encourages moves that preserve connectivity and maintain open, flexible board structures. However, the adjacency condition provides only a simplified notion of what constitutes a useful move and may overestimate mobility, particularly as reachable positions can be disrupted by future stochastic tile spawns.
[7] Completion Access	Formulation: Forming a line of length four does not always lead to a successful completion if no additional tile of the same color is available or can be moved into position. To address this, this heuristic evaluates the actionability of near-complete line structures by examining monochromatic five-cell windows and counting the number of same-color tiles that can reach the remaining positions via Breadth-First Search (BFS). Higher scores are assigned to 3-of-5 and 4-of-5 configurations with feasible completion paths, with greater emphasis on actionable four-in-a-row formations. Review: This encourages the agent to prioritize configurations that are not only structurally promising but also practically completable.

3.3.2 Delta-Based Evaluation Heuristics

Previous heuristics evaluated moves based solely on the quality of the resulting state, effectively comparing candidate boards after each action. A key innovation was to also consider the *change* in heuristic value (i.e. the delta) between the current state and the resulting state, given by

$$\Delta H = H(s') - H(s) \quad (1)$$

This captures how much a move improves the board, rather than just how good the final configuration appears in isolation. This intuition reflects practical game play, where decisions are influenced not only by the outcome of a move, but by how much it improves the current situation. This led to the development of a class of delta-based heuristics whose logic as follows in Table 2.

TABLE 2
Summary of Heuristic Evaluation Functions Designs

Heuristic	Description
Δ Gap Potential	Description: Evaluates moves based on the change in gap-based line potential. This captures the benefit of unblocking promising line structures. For example, in a configuration such as R R R B R, moving the blocking tile (B) creates a gap structure (R R R _ R), increasing the potential for immediate completion in future moves.
Δ Path Mobility	Description: Measures the change in path-based mobility. This encourages the removal of obstructing tiles that fragment the board and restrict movement. For instance, relocating a tile that blocks pathways between same-color tiles can increase their ability to connect, improving future line-building opportunities.
Δ Completion Access	Description: Evaluates how a move improves access to completion opportunities. This prioritizes moves that make near-complete lines actually completable. For example, clearing a blocking tile may allow additional same-color tiles to reach the final required position.

3.3.3 Composite Heuristic Functions

At any turn, a player often doesn't evaluate moves on any single factor, but rather considers a number of factors simultaneously into their decision-making process. This motivates the construction of a composite heuristic that combines the complementary heuristics developed previously into a single weighted evaluation function, formulated as follows:

$$H(s, s') = \alpha L(s') + \beta G(s') + \gamma M(s') + \rho P(s') \\ + \eta \Delta A(s, s') + \delta_g \Delta G(s, s') \\ + \delta_p \Delta M(s, s') + k \cdot R(s, s')$$

where $L(s')$ denotes the line potential, $G(s')$ the gap-based potential, $M(s')$ the path-based mobility, and $P(s')$ the preview alignment of the successor state s' . The terms $\Delta A(s, s')$, $\Delta G(s, s')$, and $\Delta M(s, s')$ represent the aforementioned delta heuristics. The term $R(s, s')$ corresponds to the cleared bonus heuristic. The coefficients $\alpha, \beta, \gamma, \rho, \eta, \delta_g, \delta_p$, and k represents the weightage of how much each heuristic is factored into evaluation.

3.3.4 Parameter Optimization / Grid-search

Following the construction of the composite heuristic function, a natural query is the choice of fixed weighting parameters, which determine how strongly the agent priorities each

heuristic component in its decision-making. To systematically identify effective settings, a grid search approach was employed. This involved defining a discrete set of candidate values for each parameter and exhaustively evaluating all possible combinations. Each parameter configuration was assessed based on game play performance, measured by the average score achieved over multiple runs. This process enabled the identification of parameter values that lead to higher consistency and performance.

3.4 Adaptive Heuristic Framework

While composite heuristic functions provide a clean framework for evaluating states, an evident limitation lies in the use of fixed weighting parameters. In effective human game-play, priorities and what determines a good move constantly shift significantly depending on the game phase and board condition. For example, in the early game, characterized by a relatively unoccupied board, there is greater flexibility to prioritize structural setup. In contrast, the late game, where the board is heavily occupied, places greater emphasis on moves focused about survivability, mobility, and immediate line clears to avoid early termination. This motivates the introduction of an adaptive heuristic framework, in which the evaluation function dynamically adjusts its weights depending on the current game phase and board state.

3.4.1 Dynamic Weight Adjustment

To account for these changing priorities, the composite heuristic was extended into a state-dependent adaptive framework. In trading language, this is similar to defining state features such as volatility, trend strength, liquidity, or congestion, and then adjusting the relative importance of decision factors depending on the current market condition. In the present setting, the analogue is the game board state.

The first step was therefore to define a set of state features that describe the current board condition. These included the fill ratio, line score, gap score, mobility score, preview score, access score, and the number of inaccessible fours. Based on these features, the board was then classified into hand-designed game states.

Weights were then adjusted according to these states using domain knowledge and observed game-play behavior. For example, when the board became more constrained, as signaled by predefined markers such as a fill ratio above 0.30, the weighting on heuristics such as *path-based mobility* and *completion accessibility* was increased to improve flexibility and preserve actionable configurations.

These situations and thresholds were identified through a combination of experimentation and human intuition—for instance, recognizing that mobility becomes more critical as the board fills and then formalizing this through measurable features such as fill ratio.

In this way, the evaluation function no longer relies on a single fixed set of coefficients, but instead approximates a state-dependent policy, where the relative importance of each heuristic component varies according to the current board condition. This allows the agent to reflect the changing priorities of effective play as board conditions evolve and to exhibit more context-sensitive and robust decision-making.

3.5 Machine Learning-Based Heuristic Optimization

The original heuristic framework can be interpreted as a linear combination of handcrafted features, where each feature represents a specific structural or positional property of the game state (e.g. line potential, gap potential, mobility, and accessibility). Formally, this corresponds to an approximate value function of the form:

$$V(s) = \mathbf{w}^\top \mathbf{x}(s),$$

where $\mathbf{x}(s)$ is the feature vector extracted from state s , and $\mathbf{w}$ is a vector of weights.

While the adaptive heuristic framework introduces state-dependent weighting, the weights $\mathbf{w}$ were still determined by manually specified relationships between features and defined states. While effective to a degree, this approach assumes that the optimal weighting structure is known *a priori* and is evidently limited by human intuition, possibly failing to capture more subtle interactions between features. To overcome this limitation, machine learning methods are introduced, reformulating the problem as one of learning the value function directly from experience.

3.5.1 Deep Reinforcement Learning with Actor–Critic Architecture

To achieve this, a deep reinforcement learning approach is adopted using an actor–critic architecture. Rather than operating on raw board states, the model retains the engineered feature representation and learns a nonlinear mapping from engineered features of resulting states $\mathbf{x}(s, a)$, with the objective of maximizing the expected cumulative reward over an episode. The network consists of a shared feature-processing component, followed by two outputs: an actor, which assigns a score z_a to each candidate action based on the features of the resulting state, and a critic, which estimates the value function $V(s)$. This preserves the interpretability of the handcrafted features while enabling the model to learn more expressive combinations of them.

(1) Policy learning: At each decision step, the actor defines a probability distribution over candidate actions via a softmax transformation,

$$\pi(a | s) = \frac{\exp(z_a/T)}{\sum_b \exp(z_b/T)},$$

where T is a temperature parameter controlling the degree of exploration. Learning is driven by the advantage function,

$$A(s, a) = r + \gamma V(s') - V(s),$$

which measures the relative quality of an action compared to the current value estimate. The actor is updated using the policy gradient objective

$$\mathcal{L}_{\text{actor}} = -\log \pi(a | s) \cdot A(s, a),$$

while the critic is trained to minimise the Temporal Difference error,

$$\mathcal{L}_{\text{critic}} = (r + \gamma V(s') - V(s))^2.$$

Together, these updates enable simultaneous improvement of the policy and value function.

(2) Imitation learning: Since the adaptive heuristic model was shown to exhibit strong performance, it is used as

an expert policy to accelerate learning. The actor is first pretrained using behavior cloning, where it learns to imitate expert-selected actions via a cross-entropy loss,

$$\mathcal{L}_{\text{BC}} = \text{CE}(\pi(\cdot | s), a_{\text{expert}}).$$

During reinforcement learning, this objective is retained as an auxiliary term, leading to a combined loss of the form

$$\mathcal{L} = \mathcal{L}_{\text{actor}} + \lambda_v \mathcal{L}_{\text{critic}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}},$$

where λ_v and λ_{BC} control the relative contributions of value learning and imitation. This hybrid formulation enables faster convergence by guiding the policy towards strong heuristic behavior, while still allowing it to improve further through reinforcement learning.

(3) Exploration: Exploration is achieved through stochastic sampling from the policy $\pi(a | s)$ rather than ϵ -greedy selection. The temperature parameter T provides a smooth mechanism for controlling randomness, with lower values favoring near-deterministic behavior and higher values encouraging broader exploration of the action space.

(4) Limitations: Despite its flexibility, the proposed approach introduces significant computational overhead. Furthermore, there is no guarantee of stable or consistent convergence, and performance may vary significantly depending on initialization, hyperparameters, and training dynamics. Even when convergence is achieved, it may require a considerable number of training episodes.

Overall, by combining feature-based representation, nonlinear function approximation, policy optimization, and imitation learning, the model successfully integrates the handcrafted heuristic framework and the adaptive weighting scheme with deep reinforcement learning. This enables the model to learn through self-play and continually refine its policy based on its own experience. This enables the agent to have the capacity to move beyond our adaptive heuristic model by capturing context-dependent relationships between features, rather than relying on fixed or explicitly defined weighting rules. In particular, it can learn subtle patterns in board states and recognize when to prioritize different objectives under varying conditions, much like a human player who, through repeated gameplay, develops an intuition for how to adapt decisions in nuanced and unique situations.

3.6 Lookahead Search and MCTS Strategies

While heuristic evaluation functions provide an efficient mechanism for guiding decision-making, their effectiveness is inherently limited by the depth of search. In particular, a purely greedy (depth-1) approach evaluates only the immediate outcome of actions. This limitation is commonly referred to as the *myopic heuristic problem*, where the agent fails to account for beneficial sequences of actions that only yield rewards over multiple steps.

To address this limitation, lookahead search was introduced. In this approach, the agent simulates future states beyond the immediate move, evaluating sequences of actions rather than single-step outcomes. The following subsections describe the different ways lookahead is incorporated in *Five-or-More*, including our key exploration of Monte Carlo Tree Search in relation to the research objectives.

3.6.1 Utilizing Preview Colors

Firstly, a limited form of short-horizon or shallow lookahead is incorporated through the use of the preview colors, which provide partial information about the next stochastic event. Recall in *Five-or-More*, at each step the player is given the colors of the next three tiles to be spawned, which can be leveraged to guide current decision-making.

The preview color heuristic evaluates moves based on how well the resulting configuration aligns with these upcoming tiles. For example, if two of the upcoming tiles are blue, higher value is assigned to moves that create a near-complete blue line (e.g. four consecutive blue tiles) which can be completed in the following turn by the spawned tiles. In this way, the heuristic introduces a limited form of anticipatory planning, allowing the agent to favor moves that not only improve the current state but also increase the likelihood of exploiting future tile spawns.

3.6.2 Two-Step Lookahead

A more direct form of lookahead is to explicitly consider the next move. A simplified two-step lookahead was formulated, where sequences of two consecutive actions are evaluated ignoring the game’s spawn mechanism. This allows the agent to capture interactions between moves, such as unblocking previously inaccessible tiles or repositioning pieces to enable stronger follow-up plays. The formulation is performed while *ignoring spawn*, treating the environment as temporarily deterministic across the two steps. This results in an optimistic evaluation, as it assumes that no disruptive tiles are introduced between actions.

Due to the significant computational cost of evaluating all possible two-step action sequences, a top- k pruning strategy was applied. At each step, only the top-10 candidate moves, as ranked by the heuristic evaluation, were retained for further expansion

3.6.3 Monte Carlo Simulation

Monte Carlo Tree Search can be viewed as a unifying extension of the earlier lookahead mechanisms. While the preview color heuristic incorporates partial information about the next stochastic event, and the two-step lookahead evaluates short deterministic sequences (ignoring spawn), MCTS performs multi-step simulations under the full game dynamics.

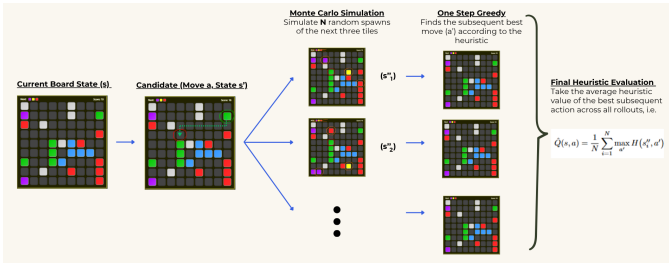


Fig. 6. Example: Monte Carlo rollout evaluation in *Five-or-More* for a single candidate move.

For each candidate move a , the agent considers the resulting board state s' obtained after applying the move. Rather than directly evaluating this state, N stochastic

rollouts are performed by sampling random spawns of the next three tiles according to the game dynamics, producing states s''_i .

For each sampled state s''_i , the agent evaluates the best subsequent action a' according to the heuristic function. The value of the original move is then computed as:

$$\hat{Q}(s, a) = \frac{1}{N} \sum_{i=1}^N \max_{a'} H(s''_i, a'),$$

which reflects the expected quality of the move under stochastic outcomes followed by a greedy one-step improvement.

(1) Simulation setting: In contrast to standard Monte Carlo methods, which typically simulate long trajectories and evaluate terminal outcomes, the approach here is restricted to a single stochastic transition followed by a greedy one-step evaluation. In principle, one could simulate further by repeatedly alternating between moves and random spawns; however, this quickly becomes intractable. After each simulated move, the next three tile spawns are unknown, and the number of possible spawn combinations grows rapidly, leading to a combinatorial explosion. As a result, exhaustively considering all possible future sequences is computationally infeasible.

Furthermore, Monte Carlo simulation is applied only in situations where no immediate clearing move is available. When a clearing move exists, its outcome is largely deterministic and dominates the evaluation, making stochastic sampling unnecessary. Limiting simulation to non-clearing states ensures that it is used only when uncertainty meaningfully influences decision-making.

(2) Top- K pruning: Similar to the approach used in the lookahead setting, the exponential growth in the number of possible action sequences necessitates pruning. To address this, top- K pruning is applied, where only the highest-ranked actions—according to the heuristic—are selected for simulation. This focuses computational effort on the most promising regions of the search space while maintaining tractability.

3.7 Verification, Validation, and Testing Methodology

The following evaluation methodology was used to assess and compare the performance of the proposed models.

3.7.1 Performance Metrics

Two primary metrics were used to evaluate the performance of each agent: the maximum score achieved and the average score across multiple runs. For the individual heuristic experiments, results were averaged over 100 runs to ensure statistical reliability. However, for the composite, adaptive, and search-enhanced models, the number of runs was reduced to 20 due to their increased computational complexity, with some individual runs taking up to an hour to complete. This reflects a balance between maintaining meaningful evaluation and ensuring computational feasibility.

The maximum score represents the agent’s peak performance, providing an estimate of its upper capability. This is particularly relevant in games such as *Five-or-More*, where the primary objective is to obtain the highest score, regardless of the number of attempts.

In contrast, the average score provides a measure of consistency and robustness, which are commonly prioritized in real-world domains such as financial trading. By aggregating performance across many stochastic, this metric provides a more reliable estimate of the agent's performance and decision making under stochastic uncertainty.

To contextualize the agent's performance, results were benchmarked against real scores achieved by human players. Logged observations from the official *Five-or-More* platform indicate that top daily scores typically range between 2000 and 10,000 points, with exceptionally strong performances exceeding this range. This provides a practical reference for evaluating agent performance, where scores within this range can be considered competitive with skilled human play.

3.7.2 Verification of Problem Abstraction

Verification of the system was carried out through incremental testing of individual components, including state transitions, move validation, line-clearing logic, and heuristic calculations. Each component was tested independently to ensure correctness before integration into the full system. In addition, the game environment was implemented using *pygame*, enabling visual inspection and interaction with the system. This allowed the implemented mechanics to be verified against the original *Five-or-More* game, ensuring that all rules, transitions, and dynamics were faithfully replicated.

3.7.3 Validation of Proposed Solutions

Validation of the proposed heuristics was performed through both qualitative and quantitative analysis.

Qualitatively, the behavior of the agent was examined to assess whether its decision-making aligned with the intended design of each heuristic. By running simulations within the *pygame* environment, specific scenarios could be observed and analyzed in detail. This included both natural gameplay and manually constructed test cases. For example, under the path-based mobility heuristic, scenarios were designed where a tile obstructed movement and fragmented the board. In such cases, the agent was expected to prioritize moves that removed the obstruction and improved connectivity. Observing the agent selecting such moves provided evidence that the heuristic was behaving as intended.

Quantitatively, validation was performed through comparative experiments across different heuristic configurations and search strategies. By analyzing how changes in heuristic design and search enhancements influenced performance metrics, it was possible to assess whether the system behaved in accordance with the intended design principles.

3.7.4 Testing Methodology

Testing was structured to reflect the intended design and role of each model. Experiments were conducted relative to selected base models, allowing the impact of individual heuristics and search enhancements to be evaluated in terms of their incremental improvement, alongside standalone performance. Experimental settings, design choices, and their justifications are introduced prior to each set of results, ensuring that evaluation is interpreted in the appropriate context for each model.

3.8 Computational Complexity and Performance Trade-off Strategies

The design of the AI agent involves an inherent trade-off between computational efficiency and decision quality. While richer heuristic evaluation and deeper search strategies can significantly improve performance, they also introduce substantial computational overhead. As a result, careful consideration was given to optimizing both the representation and the search process to maintain tractability.

3.8.1 State Representation Optimization

The initial implementation used a NumPy-based matrix representation, which while provided clarity and ease of development, became a computational bottleneck as heuristic evaluation and search complexity increased. This was replaced with a bit-board representation, where the board state is encoded using binary masks. This enables operations such as occupancy checks, neighborhood queries, and directional shifts to be performed using efficient bit-wise operations, reducing computational cost and improving evaluation speed.

3.8.2 Reachability Computation

Valid tile movement requires determining whether a path exists between source and destination cells. Due to overhead caused by explicit queue management, the initial Breadth-First Search (BFS) implementation was replaced with a bit-wise flood-fill approach, where reachable regions are iteratively expanded using bit-shift operations. This enables efficient computation of all accessible cells from a source while reducing per-move computational cost.

3.8.3 Search Space Reduction

The introduction of lookahead search and Monte Carlo simulation further increases computational demands due to the exponential growth of the search tree. To mitigate this, as aforementioned, top k-pruning was employed to restrict exploration to the most promising regions of the search space. Similarly, the number of Monte Carlo rollouts per action is bounded to ensure that computation remains feasible within practical time constraints.

3.9 Human-AI guided Gameplay Interface

As part of the extended system design, the *pygame* implementation of *Five-or-More* was augmented to support a human-AI hybrid interaction framework. This interface enables real-time switching between AI-controlled and human-controlled gameplay, allowing a human player to intervene, override decisions, or resume control at any point during execution. This was implemented through a simple control interface using keyboard inputs, where specific keys toggle between AI and human modes, pause execution, or trigger step-wise AI decisions. In human mode, players interact with the board via mouse selection, while in AI mode, moves are executed automatically according to the agent's policy. Visual feedback, including move highlighting and preview information, was incorporated to improve interpretability and facilitate analysis of decision-making behavior.

4 RESULTS

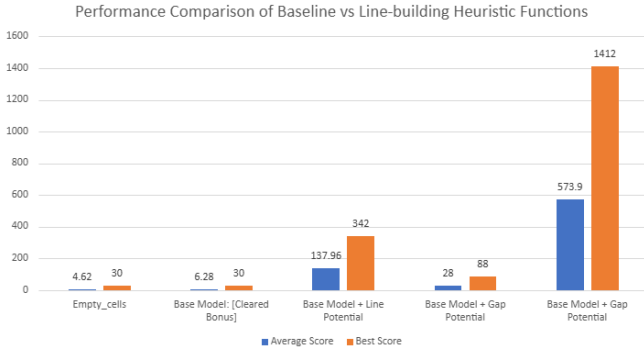
This section presents the empirical results obtained. Performance was evaluated using both average score and maximum score across multiple runs. Individual heuristic experiments were conducted over 100 runs, while composite, adaptive, and search-enhanced models were evaluated over 20 runs. **Recall** that strong human gameplay typically achieves scores in the range of 2000–10,000, providing a benchmark for evaluating agent performance globally.

4.1 Handcrafted Heuristics Performance Results

Initial experiments evaluated the proposed handcrafted heuristics using an incremental design framework. As not all heuristics are intended to operate effectively in isolation, they were grouped by functional role and introduced progressively on top of a defined base model. This approach enables the marginal contribution of each heuristic to be assessed both individually and in combination, providing a more realistic evaluation of their effectiveness within a complete decision-making system. For combined heuristics, their respective weights were adjusted such that their output values operated on comparable scales. This normalization prevented any single heuristic from dominating the evaluation due to differences in magnitude.

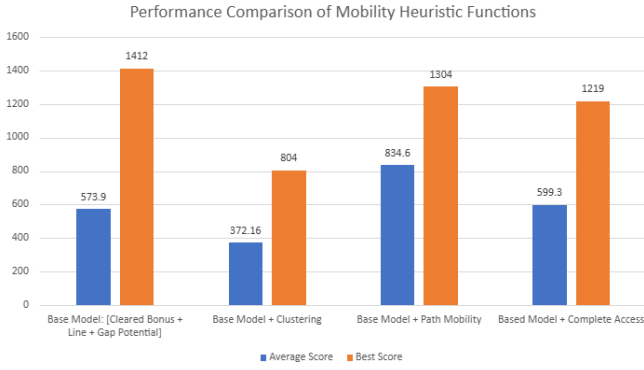
4.1.1 Line Formation Heuristics

Results are shown in Fig. 4.1.1, demonstrating introducing line-based heuristics leads to a significant improvement over baseline performance.



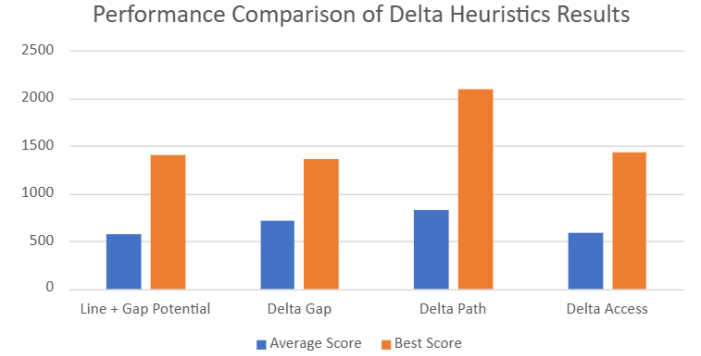
4.1.2 Mobility-Based Support Heuristics

Fig. 4.1.2 shows the performance changes resulting from the addition of mobility-based heuristics.



4.1.3 Delta-Based Heuristics

Fig. 4.1.3 shows consistent improvements over the baseline across all Δ -based heuristics, reflecting their ability to unblock configurations and increase available decision options.



4.2 Composite vs Adaptive Heuristic Framework

This section presents the direct comparison between the composite heuristic model and adaptive heuristic model.

Both models share a common functional form:

$$H(s') = \alpha \cdot \text{Line}(s') + \beta \cdot \text{Gap}(s') + \gamma \cdot \text{Mobility}(s') + \delta \cdot \Delta\text{Gap}(s') + \epsilon \cdot \Delta\text{Mobility}(s') + \eta \cdot \Delta\text{Access}(s') + k \cdot \text{ClearBonus}(s')$$

The key distinction being the composite model uses fixed weights obtained via grid search, whereas the adaptive model dynamically adjusts these weights based on predefined state-dependent conditions.

4.2.1 Grid Search Composite Heuristic Settings

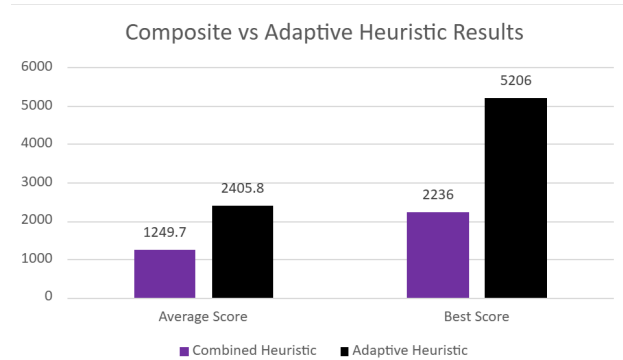
The results of grid search yielded the following parameter values for the composite heuristic model:

$$\alpha = 0.4, \beta = 0.1, \gamma = 0.005, \delta = 0.06, \epsilon = 0.001, \eta = 0.08, k = 9.$$

Clustering was excluded from the composite model as it did not demonstrate meaningful performance improvements in earlier experiments. Completion access was also omitted, as its functionality overlaps with gap-based evaluation, and including both was found to introduce redundancy without additional benefit.

4.2.2 Composite vs Adaptive Performance Results

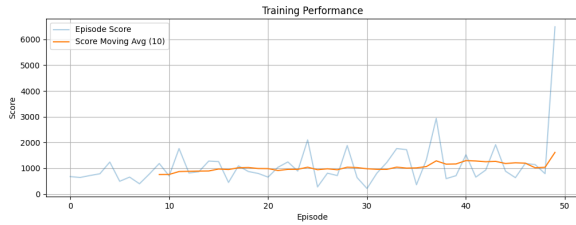
The performance of both models is shown in Fig. 4.2.2.



4.3 Deep Reinforcement Learning-Based Integration

The deep reinforcement learning model was trained using an actor-critic architecture over our engineered heuristic features. The model used a bit-board state representation and an 11-dimensional feature vector. Before reinforcement learning, the actor was pretrained for 10 games using imitation learning from the adaptive heuristic model. It was then trained for 50 episodes using a learning rate of 10^{-4} and discount factor $\gamma = 0.99$.

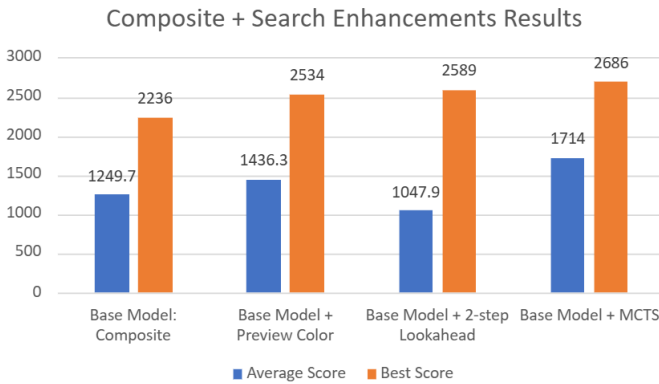
Fig. 4.3 shows the training performance of the model. Although some improvement is observed across episodes, the training curve does not show clear convergence. Performance remains variable, indicating that the model has not yet learned a consistently strong policy within the available training budget. However, the model achieved a peak score of 6488 in the final episode, surpassing the best score obtained by the adaptive heuristic model by 24.6%. While the observed improvement is promising, it remains unclear whether this reflects stable learning or stochastic variation.



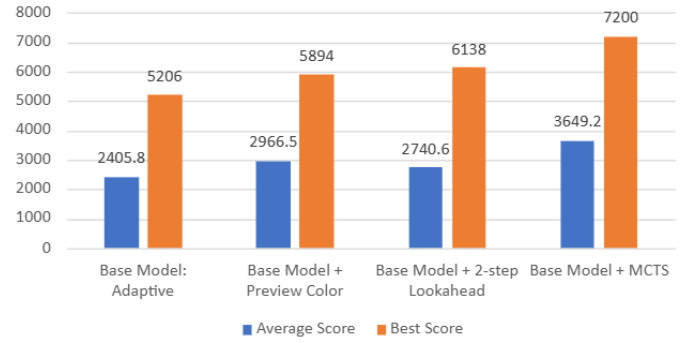
4.4 Search-Based Enhancements

Search-based enhancements, including preview color information, two-step lookahead, and Monte Carlo simulation were tested by integration into both the composite and adaptive heuristic frameworks respectively. Each enhancement was applied independently to assess its impact on decision performance.

As can be seen in Fig. 4.4 and 4.4, in both cases we see every search enhancement improves performance, especially MCTS, albeit not uniformly across models. This suggests a correlation between the quality of the underlying heuristic and the effectiveness of MCTS, with stronger heuristics enabling more meaningful evaluation of simulated outcomes.



Adaptive + Search Enhancement Results



4.5 Final Summary Results

Results are summarized in Table. 3. Several key trends emerge across all experiments. Individually designed heuristics achieve reasonable performance despite their simplicity relative to other models. A substantial improvement is observed with composite heuristics, surpassing the 2000 threshold and entering the lower bound of strong human performance (2000–10,000). Further gains are achieved with the adaptive model, indicating that increased heuristic complexity and state-dependent evaluation improve decision-making under stochasticity. Search enhancements improve both models; however, results suggest magnitude of improvement depends on the quality of the underlying heuristic. Overall, these results show that combining structured heuristics, adaptive weighting, and selective search allows for effective decision-making under uncertainty.

TABLE 3
Summary of Heuristic Performance

Model / Heuristic Configuration	Avg	Best
Baseline and Line-Formation Heuristics		
Empty Cells	4.62	30
Cleared Bonus	6.28	30
Cleared Bonus + Line Potential	137.96	342
Cleared Bonus + Gap Potential	28.00	88
Cleared Bonus + Line + Gap Potential	573.90	1412
Mobility-Based Support Heuristics		
Line + Gap + Cleared Bonus	573.90	1412
Line + Gap + Cleared Bonus + Clustering	372.16	804
Line + Gap + Cleared Bonus + Path Mobility	834.60	1304
Line + Gap + Cleared Bonus + Comp Access	599.30	1219
Delta-Based Heuristics		
Line + Gap Potential	573.90	1412
Line + Gap Potential + Δ Gap Potential	719.30	1364
Line + Gap Potential + Δ Path Mobility	830.60	2100
Line + Gap Potential + Δ Comp Access	592.80	1438
Full Model Comparisons		
Composite Heuristic	1249.7	2236
Adaptive Heuristic	2405.8	5206
Search Enhancement Comparisons		
Composite + Preview Color	1436.3	2534
Composite + Lookahead	1047.9	2589
Composite + MCTS	1714	2686
Adaptive + Preview Color	2966.5	5894
Adaptive + Lookahead	2740.6	6138
Adaptive + MCTS	3649.2	7200

5 EVALUATION

This section evaluates the performance of the proposed approaches, focusing on how different agent models, heuristic designs and search enhancements influence decision-making in the stochastic environment of *Five-or-More*.

5.1 Individual Heuristic Comparison

The incremental and ablation-style experiments results, supported by qualitative inspection of gameplay simulations, indicate that the handcrafted heuristics behaved largely in accordance with their intended design and were well aligned with the objectives of the game.

The trivial empty-cell heuristic provided only weak performance, as expected, since it does not explicitly encourage line formation or scoring.

The first major improvement came from combining cleared bonus with line potential, which allowed the agent to both set up and prioritize scoring opportunities.

Gap potential further strengthened this behavior. When combined with line potential, it produced a substantial increase in performance (approximately +316% in average score), supporting the hypothesis that it makes line setup more robust by introducing an additional line-formation strategy. This suggests that effective play requires multiple line-building strategies rather than a single rigid notion of sequence formation.

Among the mobility heuristics, adding clustering improved performance (approximately -35% in average score). This results was expected considering the game mechanics, since spatial proximity alone does not imply reachability under path constraints. Completion access produced only modest gains of +4.4% in average score, likely because it captures a similar five-window completion logic to gap potential and therefore offered less additional information. In contrast, while path-based mobility didn't improve the best score, it improved average performance by +45.4% indicating that it contributed primarily through greater consistency and reduced early board collapse. This supports its intended role as a mobility heuristic that preserves flexibility and creates future opportunities rather than directly producing clears.

The Δ -based heuristics were also effective. Their improvement over the baseline suggests that measuring board improvement, rather than only evaluating absolute successor quality, is useful in this environment. In particular, their ability to capture unblocking behavior and the creation of new opportunities proved valuable.

Overall, these results demonstrated reasonable performance and showed that effective heuristic functions can be constructed from domain knowledge, gameplay experience, and the logic of the underlying game mechanics. Even without highly complex search, such heuristics can produce strong behavior by maintaining board states that are more resilient to stochastic disruption.

5.2 Composite vs Adaptive Heuristic Framework

The composite heuristic model represented the next major improvement in performance, achieving best scores that surpassed the 2000 threshold and entered the lower bound of strong human performance (2000–10,000). This supports the

view that a good move is generally multi-faceted: effective decisions simultaneously involve multiple considerations including line formation, mobility preservation, unblocking, and immediate scoring potential. The strong performance of the composite model therefore validates the decision to combine complementary heuristic components rather than rely on any single indicator of board quality.

The adaptive heuristic framework improved performance significantly, increasing average score by approximately 92.5% and best score by approximately 132.8% compared to the composite model, thereby reaching levels competitive with strong human gameplay benchmarks. This indicates that fixed weighting alone is insufficient in stochastic environments such as *Five-or-More* and that the relative importance of heuristic signals must shift dynamically as the game evolves. Although the adaptive model does not surpass top expert human performance, it demonstrates that state-dependent heuristic adjustment is a highly effective design principle in decision making under uncertainty.

5.3 Deep Reinforcement Learning-Based Optimization

Although the reinforcement learning model did not demonstrate clear convergence within the available 50 training episodes, it achieved a highest score of 6488 in the final episode, surpassing the best score obtained by the adaptive heuristic model. This hints at the potential of learning-based approach to exceed handcrafted heuristic performance.

While performance remains variable across episodes, the training curve exhibits a slight upward trend over time, indicating some degree of learning. However, given the limited number of episodes and the inherent instability of deep reinforcement learning, it is unclear whether the model would continue to improve and converge to a stable policy or collapse. As such, the result should be interpreted cautiously. Further training, repeated runs, and increased computational resources would be required to determine whether the model can consistently outperform the adaptive heuristic.

Nonetheless, the reinforcement learning framework should be regarded as a successful integration of handcrafted heuristic design and learning-based optimization. The actor-critic architecture was able to operate over engineered features and learn non-trivial gameplay behavior, indicating that the feature representation itself was meaningful and compatible with policy learning.

These results also highlight the difficulty of the task. Learning effective decision policies in stochastic environments with delayed rewards is computationally demanding, and performance is highly sensitive to training duration, model design, and feature quality.

5.4 Search-Based Enhancements

The search-based enhancements all improved decision-making, confirming that incorporating limited future information is beneficial in this stochastic setting.

Preview-based lookahead improved both average and best score across both models, increasing average performance by approximately 14.9% in the composite model and 23.3% in the adaptive model. This supports the view that incorporating partial future information improves robustness

and survivability by better aligning present decisions with likely future opportunities.

Two-step (spawn-agnostic) lookahead showed differing effects across the two models. For the composite model, its optimistic treatment of spawns likely led to suboptimal decisions under stochastic outcomes, resulting in a decrease in average performance (approximately 16.2%), while still increasing best score (approximately +15.8%). This suggests that although the agent was able to exploit forward planning to achieve stronger peak outcomes, it also became more prone to early failures due to decisions that ignored stochastic tile placement and backfired under unfavorable spawns.

In contrast, the adaptive model benefited more consistently, with increases in both average score (approximately +13.9%) and best score (approximately +17.9%). This is likely because the adaptive heuristic's stronger state-awareness helps filter out poor optimistic sequences, mitigating the negative effects of ignoring spawn. However, performance remained less consistent compared to the preview-based enhancement, indicating that while short-horizon planning is beneficial, explicitly incorporating stochastic information yields more stable improvements.

Monte Carlo simulation produced the strongest improvement among the search enhancements, improving performance in both models. For the composite model, average and best scores increased by approximately +37.2% and +20.1% respectively, while the adaptive model saw larger gains of approximately +51.7% in average score and +38.3% in best score.

This more pronounced improvement in the adaptive model suggests a strong dependence on the quality of the underlying heuristic. Since Monte Carlo evaluation relies on the heuristic to assess sampled states, more expressive and context-aware heuristics produce more meaningful rollout evaluations. As a result, this suggests better heuristic design leads to more reliable expected value estimates, amplifying the effectiveness of Monte Carlo-based methods.

5.5 Research Objectives

Overall, the project shows that effective decision-making in *Five-or-More* depends on three interacting elements: well-designed heuristic representations, adaptive adjustment to changing state conditions, and selective search-based anticipation of future outcomes. The proposed research questions are addressed as follows.

5.6 Question 1: How does the design of heuristic evaluation functions affect the performance and behavior of an AI search agent?

The results show that heuristic design has a decisive impact. Trivial heuristics produce weak performance, while domain-informed heuristics that capture line formation, mobility, and opportunity creation yield substantial gains. Moreover, the most effective behavior emerged not from isolated heuristics, but from combining complementary signals and adapting their relative importance to the current board state. Heuristic quality also evidently influences the extent to which search-based enhancements improve performance.

5.7 Question 2: To what extent do search-based enhancements (e.g. lookahead and MCTS) improve decision-making in a stochastic environment?

The results show that search-based enhancements do improve performance, with the greatest benefit arising from methods that incorporate future stochastic outcomes more explicitly. Preview-based lookahead, deterministic two-step lookahead, and Monte Carlo simulation each contributed positively, demonstrating that reasoning beyond the immediate state is valuable in environments with significant randomness.

In particular, the effectiveness of Monte Carlo simulation was found to be strongly dependent on the quality of the underlying heuristic. More expressive and adaptive heuristics led to greater improvements with MCTS. This highlights a key interaction between heuristic design and search: search-based methods do not operate independently, but instead amplify the strengths (and weaknesses) of the evaluation function on which they rely.

5.8 Limitations

A key limitation of the handcrafted approach is that it explores only a limited portion of the possible design space, with performance dependent on how well manually designed heuristics capture strategic aspects of play. While combining heuristics improves decision quality, it also introduces computational overhead, leading to a trade-off between expressiveness and efficiency despite optimizations.

The composite model is constrained by fixed weighting, while the adaptive model, although more effective, remains limited by the quality of its predefined state conditions. If these do not fully capture the true dynamics of the board, performance may still be sub-optimal.

Search enhancements are similarly bounded by computational constraints. Limited rollouts and pruning restrict the depth and accuracy of stochastic evaluation. As a result, improvements are constrained by the approximations made to maintain tractability.

Although the models achieve competitive performance, peak scores do not surpass top human benchmarks. This is likely due to the limitations of handcrafted and predefined adaptive structures, which may fail to capture the full complexity of the environment. More advanced approaches, such as deep reinforcement learning or fully learned heuristics, offer potential to discover more nuanced and effective strategies beyond manual design.

The experimental design is also subject to several limitations. Although the chosen number of runs provides a reasonable basis for comparison given computational constraints, larger sample sizes would yield more stable estimates of average performance in a stochastic environment. Deep reinforcement learning experiments were similarly constrained by available resources, limiting the number of training episodes and preventing a fuller exploration of convergence behavior.

More broadly, the breadth of experiments was restricted by runtime considerations. The number of heuristics, model variants, and search settings that could be tested was therefore limited. As a result, while the findings are informative and internally consistent, they should not be interpreted as an exhaustive exploration of the full design space.

6 CONCLUSION

The development of AI agents for the game *Five-or-More* has provided a structured environment for exploring decision-making under uncertainty. Through this project, key insights were gained into how heuristic design, adaptive strategies, and search-based methods interact within stochastic environments. These insights extend beyond the domain of game AI, offering relevance to broader applications such as algorithmic trading, sports analytics, and decision-support systems, where uncertainty and dynamic conditions play a central role.

6.1 Key Findings

This project highlights two central insights. Firstly, heuristic design plays a critical role in determining agent performance, with no single optimal formulation. Handcrafted heuristics are inherently shaped by domain knowledge and subjective interpretation, and different perspectives on what constitutes a “good move” can lead to substantial variation in performance. Effective agents therefore emerge from combining multiple heuristics within a composite framework and adapting them dynamically to the game state.

Secondly, search-based enhancements such as Monte Carlo Tree Search (MCTS) can significantly improve decision-making by incorporating future information. However, these gains are constrained by computational cost and remain strongly dependent on the quality of the underlying heuristic. As a result, improvements from search and heuristic design are not independent, but closely intertwined.

Together, these findings emphasise the importance of balancing domain knowledge, adaptability, and computational efficiency when designing AI systems for decision-making under uncertainty.

6.2 Project Deliverable Review

Beyond autonomous gameplay, the introduction of a human-AI interactive interface adds practical value by enabling qualitative analysis, real-time testing, and collaborative decision-making. Such a design has clear parallels in domains like quantitative trading, where algorithmic systems assist human operators who retain oversight in uncertain or high-risk scenarios. The implementation demonstrates that the AI system is not only capable of autonomous decision-making but can also be effectively integrated into interactive environments that combine algorithmic efficiency with human judgment.

6.3 Applications

The *Five-or-More* environment serves as an effective abstraction of a broader class of decision-making problems under uncertainty. Despite its relatively simple rules, the game encapsulates key challenges found in more complex domains, including stochastic state transitions, limited foresight, combinatorial action spaces, and the need to balance short-term gains against long-term strategic positioning.

While each AI application requires domain-specific modelling and tailored solutions, the principles explored in this project are widely transferable. In particular, the process of designing, evaluating, and refining heuristic functions

mirrors feature engineering in fields such as quantitative trading, sports analytics, and operational research. Similarly, the integration of adaptive strategies and search-based methods reflects common approaches used in decision-support systems where uncertainty and evolving conditions must be accounted for.

The framework developed in this project—combining domain-informed heuristics, adaptive evaluation, and search-based enhancements—can therefore be viewed as a general methodology for approaching complex decision-making tasks. It highlights how structured reasoning, iterative refinement, and computational trade-offs can be systematically applied to develop effective strategies across a wide range of applications. As such, the approaches explored here provide a foundation for extending heuristic-driven and hybrid AI methods to more complex and high-stakes decision-making environments.

6.4 Future Work

Given the limitations encountered in this project, several directions for future research can be identified.

Firstly, future work could explore a wider heuristic design space, incorporating more sophisticated and diverse evaluation strategies. While this project relied on handcrafted heuristics informed by domain knowledge, an important extension would be to explore transitioning towards fully learned models using deep reinforcement learning. Such approaches would enable the agent to autonomously discover effective strategies through interaction with the environment, potentially identifying patterns and behaviours that are difficult to encode manually.

Secondly, the relationship between heuristic design and search performance warrants further investigation. As demonstrated, the effectiveness of search methods such as Monte Carlo Tree Search (MCTS) is closely tied to the quality of the underlying evaluation function. Future research could therefore explore alternative MCTS variants, improved rollout policies, and hybrid approaches, alongside more efficient search strategies that better balance computational cost with performance gains.

Additionally, increased computational resources could allow for deeper search, more extensive simulations, and more robust evaluations. This would further support the development of more scalable and high-performing agents.

While the proposed approaches did not surpass top expert human performance, they were able to achieve scores within the competitive human benchmark range. Future work could aim to bridge this gap by improving adaptive representations or leveraging fully learned models capable of capturing more nuanced and complex strategies beyond handcrafted design.

Finally, practical extensions of this work could include integrating the system with the official game interface, enabling real-time analysis and decision support during live gameplay. This would enhance the applicability of the system as an interactive tool and provide a more realistic demonstration of its capabilities.

Overall, these directions highlight the potential to further improve both the performance and applicability of the proposed AI system, while continuing to explore the broader challenges of decision-making under uncertainty.

REFERENCES

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.
- [2] R. E. Korf, "Heuristic evaluation functions in artificial intelligence search algorithms," *Minds and Machines*, vol. 5, no. 4, pp. 489–498, 1995.
- [3] D. S. Johnson and L. A. McGeoch, "The traveling salesman problem: A case study in local optimization," in *Local Search in Combinatorial Optimization*, E. H. L. Aarts and J. K. Lenstra, Eds. John Wiley & Sons, 1997, pp. 215–310.
- [4] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis, "A general reinforcement learning algorithm that masters chess, shogi, and go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018. [Online]. Available: <https://arxiv.org/abs/1712.01815>
- [5] A. W. Lo, "The adaptive markets hypothesis: Market efficiency from an evolutionary perspective," *Journal of Portfolio Management*, vol. 30, no. 5, pp. 15–29, 2004.
- [6] M. Mikołajek-Gocejna, "Heuristic: Psychological aspects of decision-making on capital market," *Zeszyty Naukowe Uniwersytetu Szczecińskiego. Finanse, Rynki Finansowe, Ubezpieczenia*, no. 89, pp. 2–11, 2017.
- [7] R. M. Galekwa, J. M. Tshimula, E. G. Tajeuna, and K. Kyandoghere, "A systematic review of machine learning in sports betting: Techniques, challenges, and future directions," *arXiv preprint arXiv:2410.21484*, 2024.
- [8] I. Bucak and V. Uslan, "An analysis of sequence alignment: Heuristic algorithms," in *Proceedings of the Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBS)*, 2010, pp. 5626–5628.
- [9] L. Yakovlev and L. Chechurin, "Creativity and heuristics in process control engineering," *Chemical Engineering Research and Design*, vol. 103, pp. 40–49, 2015.
- [10] M. Josifoski, M. Peyrard, F. Rajcic, J. Wei, D. Paul, V. Hartmann, B. Patra, V. Chaudhary, E. Kicman, B. Faltings, and R. West, "Language model decoding as likelihood–utility alignment," *arXiv preprint arXiv:2210.07228*, 2023.
- [11] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.
- [12] T. D. Lab, "Markov decision processes," 2023, accessed: 2026-04-27. [Online]. Available: <https://thedecisionlab.com/reference-guide/mathematics/markov-decision-process>
- [13] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [14] W. Eyinade, O. J. Ezeilo, and I. A. Ogundeyi, "Financial risk management strategies and their influence on organizational stability," *Finance & Accounting Research Journal*, vol. 7, no. 6, pp. 229–251, 2025.
- [15] N. J. Nilsson, *Principles of Artificial Intelligence*. Morgan Kaufmann, 1980.
- [16] U. Berkeley, "Cs188: Introduction to artificial intelligence - search," 2023, accessed: 2026-04-27. [Online]. Available: <https://inst.eecs.berkeley.edu/~cs188/>
- [17] J. Clune, "Heuristic evaluation functions for general game playing," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2007.
- [18] V. Allis, "A knowledge-based approach of connect-four," *Master's Thesis, Vrije Universiteit Amsterdam*, 1988.
- [19] P. Liashchynskiy and P. Liashchynskiy, "Grid search, random search, genetic algorithm: A big comparison for nas," *arXiv preprint arXiv:1912.06059*, 2019. [Online]. Available: <https://arxiv.org/abs/1912.06059>
- [20] D. B. et al., "The challenge of poker," *Artificial Intelligence*, vol. 134, no. 1-2, pp. 201–240, 2002.
- [21] G. Hayman and R. K.-K. Pang, "Reframing financial markets as complex systems: Tools for systemic risk analysis, portfolio management, and system-level investing," CFA Institute Research and Policy Center, Tech. Rep., 2025. [Online]. Available: https://rp.cfa-institute.org/sites/default/files/docs/research-reports/rpc_hayman_complexsystemsforinvestmentmanagement_online.pdf
- [22] MLQ.ai, "The rise of alternative data in machine learning in finance," <https://blog.mlq.ai/rise-of-alternative-data-machine-learning-in-finance/>, 2023, accessed: 2026-04-27.
- [23] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning," *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.
- [24] C. B. Browne, E. Powley, D. Whitehouse, S. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, "A survey of monte carlo tree search methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.
- [25] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, pp. 484–489, 2016.
- [26] A. O. Aydınhan, X. Li, and J. M. Mulvey, "Solving multi-period financial planning models: Combining monte carlo tree search and neural networks," *arXiv preprint arXiv:2205.07347*, 2022. [Online]. Available: <https://arxiv.org/abs/2205.07347>
- [27] E. Vittori, A. Likmeta, and M. Restelli, "Monte carlo tree search for trading and hedging," in *Proceedings of the 1st ACM International Conference on AI in Finance (ICAIF)*, 2021.
- [28] P. Glasserman, *Monte Carlo Methods in Financial Engineering*. New York: Springer, 2004.
- [29] A. C. Constantinou and N. E. Fenton, "Using bookmaker odds to predict the final result of football matches," *Journal of Quantitative Analysis in Sports*, vol. 10, no. 1, pp. 1–15, 2014.
- [30] R. M. Galekwa, J. M. Tshimula, E. G. Tajeuna, and K. Kyandoghere, "A systematic review of machine learning in sports betting: Techniques, challenges, and future directions," *arXiv preprint arXiv:2410.21484*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.21484>
- [31] OddsMatrix, "Machine learning in sports betting," 2023, accessed: 2026-04-28. [Online]. Available: <https://oddsmatrix.com/machine-learning-sports-betting/>
- [32] J. Wang, Z. Zhao, and L. Wang, "Integrated financial risk management framework: A quantitative portfolio optimization analysis," *Journal of Computational Science*, 2026.
- [33] Berenberg, "Navigating volatility: A quest for superior long-term returns," 2025, accessed: 2026-04-28. [Online]. Available: <https://www.berenberg.de/en/news/multi-asset/navigating-volatility-a-quest-for-superior-long-term-returns/>
- [34] likeawizard, "Review of different board representations in computer chess," 2020, accessed: 2026-04-28. [Online]. Available: <https://lichess.org/@/likeawizard/blog/review-of-different-board-representations-in-computer-chess/S9eQCAWA>
- [35] A. S. Darabant and L. Darabant, "Clustering methods in data fragmentation," *Studia Universitatis Babeş-Bolyai, Informatica*, vol. 54, no. 1, pp. 45–60, 2009.